

Code Explanation

Delta Live Tables Code Explanation

The provided code is a Delta Live Tables (DLT) pipeline written in Python, utilizing Apache Spark and the Delta Lake format. It is designed to ingest, process, and transform customer and order data from CSV files into a series of tables with varying levels of data quality and aggregation.

Overview of the Code

The code defines a series of DLT tables that represent different stages of data processing, from raw data ingestion to aggregated insights. It implements data quality checks, data cleansing, and Slowly Changing Dimension (SCD) Type 2 logic to track changes in customer data over time.

Step 1: Importing Necessary Libraries and Defining Source Data Paths

The code begins by importing the necessary libraries, including ``pyspark.sql.functions``, ``pyspark.sql.window``, ``delta.tables``, and ``dlt``. It then defines the source data paths for customer and order data.

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from delta.tables import DeltaTable
import dlt

# Define source data paths
CUSTOMER_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata"
ORDER_PATH = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata"
```

Step 2: Defining Catalog and Schema

The code defines the catalog and schema for the DLT tables.

```
# Define catalog and schema
CATALOG = "gen_ai_poc_databrickscoe"
SCHEMA = "sdlc_wizard"
```

Step 3: Creating Bronze Tables for Raw Data Ingestion

The code creates two bronze tables, ``customer_bronze`` and ``order_bronze``, to ingest raw customer and order data from CSV files.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from source"
)
def customer_bronze():
    return (
```

```

        spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(CUSTOMER_PATH)
    )

@dlt.table(
    name="order_bronze",
    comment="Raw order data from source"
)
def order_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(ORDER_PATH)
    )

```

Step 4: Creating Silver Tables for Cleaned Data

The code creates two silver tables, `customer\_silver` and `order\_silver`, to clean and transform the raw data. The `customer\_silver` table removes nulls and duplicates, while the `order\_silver` table calculates the total amount for each order.

```

@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with nulls and duplicates removed"
)
def customer_silver():
    return (
        dlt.read("customer_bronze")
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )

@dlt.table(
    name="order_silver",
    comment="Cleaned order data with total amount calculated"
)
def order_silver():
    return (
        dlt.read("order_bronze")
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
    )

```

```

        .dropDuplicates(["OrderId"])
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
    )
)

```

Step 5: Creating ordersummary Table with SCD Type 2 Logic

The code creates an `ordersummary` table that joins customer and order data and implements SCD Type 2 logic to track changes in customer data over time.

```

@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table joining customer and order data",
    table_properties={
        "quality": "silver",
        "delta.enableChangeDataFeed": "true"
    }
)
def ordersummary():
    # Check if the table exists
    try:
        current_data = dlt.read_stream("ordersummary")
        exists = True
    except:
        exists = False

    # Join customer and order data
    joined_data = (
        dlt.read("customer_silver")
        .join(
            dlt.read("order_silver"),
            "CustId",
            "inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date"
        )
    )

    if not exists:
        return (
            joined_data
            .withColumn("IsActive", F.lit(True))
            .withColumn("StartDate", F.current_timestamp())
            .withColumn("EndDate", F.lit(None).cast("timestamp"))
        )
    else:
        # Implement SCD Type 2 logic for updates
        customer_changes = dlt.read("customer_silver")
        records_to_update = (
            current_data
            .join(
                customer_changes,
                "CustId",
                "inner"
            )
        )

```

```

        )
        .filter(
            (current_data["Name"] != customer_changes["Name"]) |
            (current_data["EmailId"] != customer_changes["EmailId"]) |
            (current_data["Region"] != customer_changes["Region"])
        )
        .select(current_data["*"])
    )

    expired_records = (
        records_to_update
        .withColumn("IsActive", F.lit(False))
        .withColumn("EndDate", F.current_timestamp())
    )

    new_records = (
        joined_data
        .join(
            records_to_update.select("CustId").distinct(),
            "CustId",
            "inner"
        )
        .withColumn("IsActive", F.lit(True))
        .withColumn("StartDate", F.current_timestamp())
        .withColumn("EndDate", F.lit(None).cast("timestamp"))
    )

    return (
        current_data
        .filter(~F.col("CustId").isin([r["CustId"] for r in records_to_update.collect()]))
        .union(expired_records)
        .union(new_records)
    )

```

Step 6: Creating customeraggregatespend Table for Aggregated Insights

The code creates a `customeraggregatespend` table that aggregates customer spending by name and date.

```

@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date"
)
def customeraggregatespend():
    return (
        dlt.read("ordersummary")
        .join(
            dlt.read("order_silver").select("OrderId", "TotalAmount"),
            "OrderId",
            "inner"
        )
        .filter(F.col("IsActive") == True)
    )

```

```
.groupBy("Name", "Date")  
  .agg(F.sum("TotalAmount").alias("TotalAmount"))  
)
```